WHAT REVEALS 3D ARTICULATION? LEARNING FROM MOTION

Anonymous authors
Paper under double-blind review

Figure 1: **MOTIONART** directly infers the underlying articulated structure of an object from two distinct states, performing the state-of-the-art in terms of both accuracy and generalization.

ABSTRACT

We present MOTIONART, a feed-forward model that directly predicts all articulations of an object, including part segmentation, kinematic structure, motion type, and continuous joint parameters, from two distinct states of the same object. Unlike existing *single*-state feed-forward methods, MOTIONART exploits the observed state change as direct evidence of articulation, by learning to match the corresponding motion across the *two* states, rather than relying heavily on category-level priors learned from limited articulated-object annotations. MOTIONART is a Siamese motion-based articulation transformer, along with a cycle-consistency training objective that encourages consistent predictions under reversed state order. We demonstrate the importance of the two-state formulation and cycle-consistency training for learning feed-forward articulations through extensive experiments. We also show that MOTIONART achieves state-of-the-art performance on articulated structure recovery, with better generalization to novel object categories and more complex kinematic layouts.

1 INTRODUCTION

How do we perceive the articulated structure of an object? Consider a child playing with an Optimus Prime robot [1]. Even without knowing its mechanism in advance, the child can quickly infer which parts move together, which joints rotate, and how one configuration transforms into another. This ability does *not* come from treating objects as static collections of isolated elements, but from observing the object across different states and reasoning about the *motion* between them.

In this paper, we thus consider the problem of inferring the articulated structure of an object from *two distinct states* of the same object, rather than guessing it from a *single static* observation. Solving this task is particularly important for enabling AI systems to understand and interact with

[1] Modern Transformer toys can contain more than 30 servomotors, with more than 100 moving parts.

functional objects in our daily lives, such as drawers, laptops, and glasses, with broad applications in robotics Xiang et al. (2020); Li et al. (2024), virtual reality Chen et al. (2025a), and embodied AI Puig et al. (2023); Li et al. (2021).

Recently, several learning-based methods have been proposed to learn a feed-forward model that can directly infer part geometry and kinematics from visual inputs, such as a single image Chen et al. (2024; 2025b) or a single 3D asset Che et al. (2024); Goyal et al. (2025); Li et al. (2026b). However, these methods still generalize poorly to unseen categories, partly because they operate in a *single-state* regime that learns the *category-specific* kinematic priors from annotated training data. However, current datasets with dense articulated annotations are extremely limited in scale and diversity [2]. In stark contrast, the *two-state* observation of the same object directly reveals the underlying motion and its articulation, which is far easier to obtain at scale than dense kinematic annotations: they can be collected from internet videos, recorded through simple manual interactions Huang et al. (2026), or synthesized by modern multimodal foundation models, like Seedance 2.0 Seedance et al. (2026), Nano Banana Pro Google DeepMind (2025), and ChatGPT image 2.0 OpenAI (2026).

Drawing inspiration from this observation, we introduce MOTIONART, a novel paradigm that infers an object's articulation from two *motion* states in a feed-forward manner. With this *two-state* learning paradigm, the task is formulated as learning to perform *corresponding motion part matching* between the two states to identify the articulated parts with underlying motion and infer the kinematic structure, unlike the previous data-driven *one-state* learning paradigm Che et al. (2024); Goyal et al. (2025); Li et al. (2026b) that learns the kinematic priors from annotated training data. Such a formulation is more intuitive and reduces the task's learning difficulty, enabling our model to achieve state-of-the-art performance and strong cross-dataset generalization ability on unseen categories.

In principle, several recent optimization-based methods Liu et al. (2023); Chen et al. (2025a); Ai et al. (2026) also exploit multi-state geometry to infer articulation (details shown in Table 3), but they typically require test-time optimization and are therefore expensive and difficult to deploy. In contrast, MOTIONART infers the full articulations in a single forward pass, in seconds, including part segmentation, kinematic hierarchy, motion type, and motion range. Architecturally, it is a Siamese query-based transformer that takes two states as input and performs bidirectional information exchange between the two branches to capture the motion transformation. To further align the model with physical laws, we introduce a cycle-consistency loss that enforces the model to produce consistent predictions when the order of the two states is swapped.

To summarize, we make the following contributions: (1) We introduce MOTIONART, a novel feed-forward paradigm that learns to infer the full articulated structure of an object from *two-state* observations. (2) We demonstrate that the two-state design allows the model to easily predict articulated structure with state-of-the-art performance on both in-domain and out-of-domain benchmarks, and we strengthen this effect with a Siamese cycle-consistency training framework and objective. (3) We also show that the additional state can be obtained cheaply in practice, like casual users' recorded media or current powerful multimodal foundation models generated images, making MOTIONART usable in real-world applications.

2 RELATED WORK

As summarized in Table 3, we divide existing articulated reconstruction approaches into three types: optimization-based, learning-based, and generation-based methods.

Optimization-based articulated reconstruction. An important line of work studies articulated objects through per-instance reconstruction from multiple observations. Early methods Liu et al. (2023); Tseng et al. (2022); Wei et al. (2022) recover geometry and articulation by fitting a deformable NeRF Mildenhall et al. (2020) to a specific object observed across multiple views or articulation states. More recent dynamic reconstruction methods Liu et al. (2025c); Wu et al. (2025a); Ai et al. (2026); Shen et al. (2025); Liu et al. (2025b); Guo et al. (2026) further exploit motion videos or continuous interaction to disentangle moving parts via Gaussian Splatting Kerbl et al. (2023). However, they are still optimization-intensive pipelines whose quality depends on how well the observations cover the

[2] Particulate Li et al. (2026b) combines PartNet-Mobility Mo et al. (2019) and GRScenes Wang et al. (2024), but still yields only 3,800 objects across 50 categories, most of which are furniture.

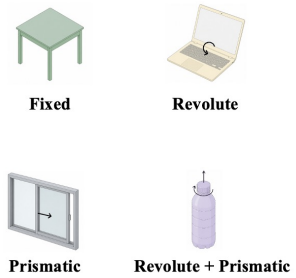

Figure 2: Articulation types.

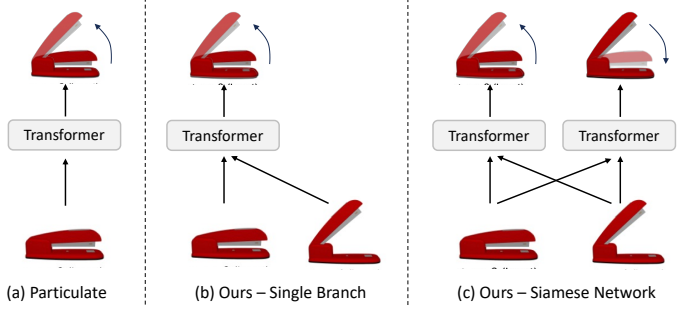

Figure 3: The different pipelines.

scene. Therefore, a generalizable end-to-end articulation estimation is what physical-world modeling really needs: not only for fast inference but also for a full understanding of physical cues.

Learning-based articulated reconstruction. Recent learning-based methods usually predict articulated structure from a single image Chen et al. (2024); Li et al. (2026a) or a single 3D observation Che et al. (2024); Goyal et al. (2025); Li et al. (2026b); Gao et al. (2025); Song et al. (2025); Li et al. (2025). For example, Particulate Li et al. (2026b) takes a single point cloud as input, and outputs the part segmentation of the point cloud and the kinematic parameter of those parts. Compared with per-instance optimization, these feed-forward methods are much more scalable and can perform inference directly and quickly. However, these methods also have limitations. Although they curate diverse articulation datasets for training, their generalization remains unsatisfactory, even within the same category. We argue that this is because they use only a single static state as conditioning for predicting action-based articulation, which makes the task itself ambiguous: a single observation may correspond to different articulation patterns, even within the same category. Therefore, we propose using multi-state observations as conditioning and designing an action-wise framework to resolve the ambiguity.

Generation-based articulated reconstruction. Another direction focuses on constructing articulated assets by retrieval, composition, or generation rather than directly inferring articulation from visual observations. These methods Liu et al. (2025a); Lei et al. (2023); Liu et al. (2024); Chen et al. (2025a;b); Wu et al. (2026); Su et al. (2025) assemble reusable parts, invoke articulation-aware priors, or combine generation with structural reasoning to produce simulation-ready articulated assets. For example, URDF-Anything+ takes an image and a mesh as inputs and leverages DiT Peebles & Xie (2023); Chen et al. (2025c) to autoregressively generate articulated objects. Recent multimodal frameworks Wang et al. (2025); Qiu et al. (2025); Lu et al. (2025); Wu et al. (2025b) further advance articulated asset generation using generative priors from VLMs Hurst et al. (2024); Google DeepMind (2025). Despite their impressive generalization in open-world scenes, they suffer from slow inference, LLM illusions, and difficulty handling complex topology.

3 METHOD

We introduce MOTIONART, a Siamese transformer architecture for learning part segmentation and articulation parameters from two states (s_0 and s_1) of an articulated object. We first formulate the problem setting in Section 3.1, followed by our Siamese transformer architecture in Section 3.2 and its multi-task decoder in Section 3.3. Finally, we describe the training objectives in Section 3.4.

3.1 PROBLEM FORMULATION

The input is two distinct physical states of an object, representing the articulation motion from start s_0 to end s_1. Specifically, the observations are two unorganized point clouds $\{\mathcal{P}^{(s_i)}\}_{i=\{0,1\}}$, where each $\mathcal{P}^{(*)} = \{\boldsymbol{p}_i^{(*)} \in \mathbb{R}^3\}_{i=1}^{N}$ is sampled from two states of the object mesh $\{\mathcal{M}^{(s_i)}\}_{i=\{0,1\}}$, respectively, where N is the number of sampled points. Our goal is to learn a Model $f_{\boldsymbol{\theta}}$ that maps these two states

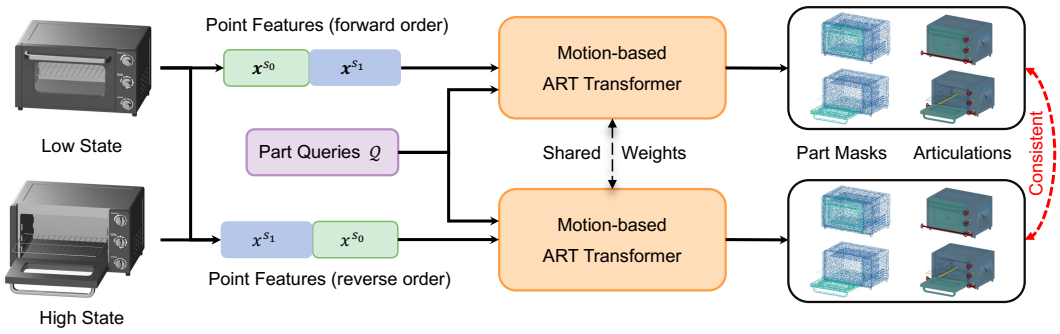

Figure 4: **Overview of MOTIONART.** Given two observed states of the same object, our framework jointly reasons over cross-state geometry to predict part segmentation and the underlying kinematic structure. We use a set of shared queries to aggregate two-state features, with each query representing a rigid part. The overall Siamese framework consists of two branches: forward order and reverse order. Each branch predicts both part segmentation of two states and articulation parameters.

of point clouds to a set of part segmentation masks and articulation parameters:

$$f_{\boldsymbol{\theta}} : \{\mathcal{P}^{(s_i)}, \mathcal{N}^{(s_i)}, \boldsymbol{F}^{(s_i)}\}_{i=\{0,1\}} \mapsto \{M^{(s_i)}, \mathcal{G}^{(s_i)}, \Theta^{(s_i)}\}_{i=\{0,1\}}, \tag{1}$$

where $\mathcal{N}^{(s_i)}$ and $\boldsymbol{F}^{(s_i)}$ are the surface normal and part features from off-the-shelf extractors of the point cloud from state s_i, respectively.

We predict all outputs for both states. However, to simplify the notation, we will only describe the output of one state, while the other state follows the same procedure. We predict the segmentation masks $M \in \{0,1\}^{N\times K}$ for K *kinematic parts* and infer the underlying *kinematic graph* $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where node $\mathcal{V}$ composed by K kinematic parts and edge $\mathcal{E}$ is a directed *kinematic tree* $J_{K\times K}$ that defines the joint connectivity between parts. We also predict the *articulation parameters* of joints $\Theta = \{\mathcal{A}, \mathcal{R}, \mathcal{T}\}$, where $\mathcal{A}$ denotes the articulation types, which are predefined as four types: {fixed, *revolute*, *prismatic*, and *revolute + prismatic*}, as shown in Figure 2; $\mathcal{R} = \{(\boldsymbol{\omega}_e, \boldsymbol{q}_e, \alpha_e)\}_{e\in\mathcal{E}}$ denotes the revolute parameters, where $\boldsymbol{\omega}_e \in \mathbb{S}^2$ is a unit rotation-axis direction, $\boldsymbol{q}_e \in \mathbb{R}^3$ is a pivot point on that axis, and $\alpha_e \in \mathbb{R}$ is the range $(\alpha_{\min}, \alpha_{\max})$ of the rotation angle between two states; and $\mathcal{T} = \{(\boldsymbol{d}_e, l_e)\}_{e\in\mathcal{E}}$ denotes the prismatic parameters, where $\boldsymbol{d}_e \in \mathbb{S}^2$ is a unit translation direction and $l_e \in \mathbb{R}$ is the range $(l_{\min}, l_{\max})$ of the translation distance between two states.

***Multi*-state *vs. single*-state.** We implement the model $f_{\boldsymbol{\theta}}$ as a feed-forward Siamese transformer architecture with shared weights, and $\boldsymbol{\theta}$ are the learnable parameters. Prior feed-forward methods Chen et al. (2024; 2025b); Che et al. (2024); Goyal et al. (2025); Li et al. (2026b) usually optimize $\boldsymbol{\theta}$ with a single state as input. This is partly reliable if the training data is sufficiently large and diverse in the deep learning era, but it is *not* the case in practice, especially for kinematic annotated 3D. We instead consider a new *feed-forward* paradigm that uses *two-state* of an object as input to learn the underlying physical rules of articulation from the state motion change, which is more efficient and generalizable than learning priors from a single state. The *core insight* is that the two states of the same object directly reveal the articulation attributes, while two-state observations are also easily obtained in real-world applications, *e.g.*, by scanning or capturing an object before and after interaction. Notably, we do *not* assume any point correspondence between two states, which is more challenging but also more practical in real-world applications. We select *two*-state because it is the minimum number of states that can reveal the articulation attributes, but our design is *not* specific to two states and can be easily extended to more states if needed.

3.2 TWO-STATE MOTION-BASED ARTICULATION TRANSFORMER

At a high level, MOTIONART is a Siamese transformer architecture with shared weights that takes two states of point clouds as input, and learn the underlying physical rules of articulation from the state motion change, rather than learning articulation priors from a single state as in prior feed-forward methods Li et al. (2026b; 2025); Wu et al. (2026). The key insight of our design is that we can

simultaneously infer full articulation attributes for two states, using only one set of shared part queries, and achieve more robust, generalizable performance by enforcing cycle consistency between them.

However, this architecture design is non-trivial because: (1) The two states of point clouds are randomly sampled and unorganized, *i.e.*, there are no matching points between the two states. [3] (2) The motion of articulated parts makes the point correspondence more complex, *i.e.*, those points of the moving parts are not even spatially aligned. The missing point correspondence indicates that although the additional state provides richer articulation cues, the network still requires careful design to fully utilize them. To address this issue, we develop a dedicated two-state Motion-based Articulation Transformer with a Siamese architecture.

Siamese Architecture with Shared Part Queries. As shown in Figure 3 (a), prior feed-forward methods predict full articulation parameters by learning from a single state of an object. However, our model includes two states of an object as input. We could instead train the network by directly concatenating the two states with $2N$ points as input and predicting the full articulation attributes for both states at the same time (Figure 3 (b)). While this naïvely concatenation works by learning with two states, it does not explicitly enforce the part queries to learn the same semantic and kinematic part across two states. More importantly, since we argue the *two-state* learning paradigm learns articulation in a way that aligns with physical laws, as it captures the underlying motion between two states rather than the instance category. Hence, if we swap the order of the observed states, the model should still produce correct and consistent predictions. We achieve this by designing a Siamese architecture with shared weights and part queries (Figure 3 (c)). Enforcing consistency between two branches using the cycle consistency loss yields more robust and generalizable performance, as shown in our experiments. Our transformer architecture is inspired by Particulate Li et al. (2026b), but features several key differences that allow us to train with two states and ensure the consistency between the two branches.

For each **input** point $\boldsymbol{p}_i \in \mathcal{P}$, we still concatenate its coordinates, normals, and part features in the channel dimension to obtain the input point features $\boldsymbol{x}_i = \left[\boldsymbol{p}_i, \boldsymbol{n}_i, \boldsymbol{f}_i\right] \in \mathbb{R}^{3+3+d}$. But now we have two states of points that are concatenated in the token dimension, *i.e.*, $\{\boldsymbol{x}_i\}_{i=1}^{2N}$. In our final reverse-order modal (Figure 4 (c)), we simply swap the order of two states of input point features.

For the **part queries**, we still use a set of shared learnable queries $\mathcal{Q} \in \mathbb{R}^{K_{\max} \times D}$, where $K_{\max}$ is set to be particularly large to cover the maximum number of parts in the training data, as the actual number of parts is unknown a priori, and D is the query dimension.

The **attention blocks** is a sequence of self-attention on part queries, followed by query-to-point attention and point-to-query attention. Here, we apply the masked attention mechanism Cheng et al. (2022) to the query-to-point attention, which first predicts a binary mask for each query to indicate the activated points, and then uses the predicted mask to modulate the attention weights in the next layer of attention. The detailed description is provided in Section E. This masked attention mechanism is crucial for our model to learn from two states without point correspondence, as it forces the part queries to focus on the activated areas of the same part across two states. After several layers of attention, we obtain the updated part queries $\tilde{\mathcal{Q}} \in \mathbb{R}^{K_{\max} \times D}$, and point tokens $\tilde{\mathcal{P}} \in \mathbb{R}^{2N \times D}$.

3.3 Prediction Heads

Following Particulate Li et al. (2026b), we use several MLPs to predict the part segmentation M, the kinematic graph $\mathcal{G}$, and the articulation parameters Θ. Note that we primarily describe the decoding process for the forward-order branch, while the other branch follows a similar procedure.

We use an MLP to predict the **point segmentation mask** $\hat{M} = \mathrm{MLP}(\tilde{\mathcal{Q}}, \tilde{\mathcal{P}})$, where $\hat{M} \in \mathbb{R}^{2N \times K_{\max}}$, which indicates the probability of part segmentation for each point. Note that it contains points from both states, even in one branch.

We predict the **kinematic graph** $\hat{\mathcal{G}}$, which is represented as a kinematic tree $\hat{J} \in \{0,1\}^{K_{\max} \times K_{\max}}$, by applying an MLP to every ordered query pair $\hat{J}_{ij} = \mathrm{MLP}(\tilde{Q}_i, \tilde{Q}_j)$, where $\tilde{Q}_i, \tilde{Q}_j$ correspond to the corresponding query of part i and j, respectively. Following the URDF definition, we designate

[3] Note that we deliberately did not perform consistent sampling of the point clouds to generate data samples, because it is too difficult to obtain the matching points in the second state as input in practical applications.

one basic part as the root node of the kinematic tree and restrict each part to have at most one parent node. Therefore, given actual K parts, there are exactly $K-1$ joints, where $K \leq K_{\max}$.

We use several MLPs to predict the **articulation parameters** $\hat{\Theta} = \text{MLPs}(\tilde{\mathcal{Q}})$, where $\hat{\Theta} = \{\hat{\mathcal{A}}, \hat{\mathcal{R}}, \hat{\mathcal{T}}\}$. Note that, separate MLPs are used to predict different types of parameters in $\hat{\Theta}$. As mentioned in Section 3.1, we predefine four types of articulation. The articulation type $\hat{\mathcal{A}}$ is predicted by a classification MLP to output the logits of query part i belonging to each of the *four* types. The revolute parameters $\hat{\mathcal{R}} = \{\hat{\boldsymbol{\omega}}_e, \hat{\boldsymbol{q}}_e, \hat{\alpha}_e\}_{e\in\mathcal{E}}$ and prismatic parameters $\hat{\mathcal{T}} = \{\hat{\boldsymbol{d}}_e, \hat{\boldsymbol{q}}_e\}_{e\in\mathcal{E}}$ are predicted by regression MLPs for each joint e. Note that the predicted parameters of a joint are only valid if the corresponding part is classified as a moving part, *i.e.*, revolute, prismatic, or revolute + prismatic.

3.4 TRAINING

During training, we sample two point cloud states and their corresponding ground-truth annotations. Note that, while we have different states, the kinematic tree, articulation type, and joint parameters should be the same for both states. We only swap the part segmentation masks between the two states, and also flip the ranges of revolute and prismatic motion to align the motion directions. This design enforces the model to learn the same kinematic structure and motion attributes across two states.

Part Matching with Ground Truth. We use the Hungarian Matching to build part correspondence following Li et al. (2026b); Carion et al. (2020). The difference with Li et al. (2026b) is that since we have two sets of output queries that predict two-state part segmentation, we only perform Hungarian Matching for the rest state and enforce the articulated state following the same matching orders. In this way, we ensure that the two output queries predict consistent part segmentation.

Training Losses. We train the MOTIONART end-to-end using a multi-task loss:

$$\mathcal{L} = \underbrace{\mathcal{L}^{\text{f}}_{\text{seg}} + \mathcal{L}^{\text{f}}_{\text{art}}}_{\text{forward branch loss}} + \underbrace{\mathcal{L}^{\text{r}}_{\text{seg}} + \mathcal{L}^{\text{r}}_{\text{art}}}_{\text{reverse branch loss}} + \underbrace{\lambda_{\text{cycle}}\mathcal{L}_{\text{cycle}}}_{\text{cycle loss}}, \tag{2}$$

where $\mathcal{L}^{*}_{\text{seg}} = \sum_{i=\{0,1\}} \text{CE}(\hat{M}^{(s_i)}, M^{(s_i)})$ is the cross entropy loss for **part segmentation** for both states s_i in branch $*$, $\mathcal{L}^{*}_{\text{art}} = \text{BCE}(\hat{\mathcal{J}}, \mathcal{J}) + \text{CE}(\hat{\mathcal{A}}, \mathcal{A}) + \ell_1(\hat{\mathcal{R}}, \mathcal{R}) + \ell_1(\hat{\mathcal{T}}, \mathcal{T})$ is the **articulation loss** for branch $*$, which includes binary cross-entropy loss for the kinematic tree $\mathcal{J}$, cross-entropy loss for the articulation type $\mathcal{A}$, and L1 losses for the revolute parameters $\mathcal{R}$ and prismatic parameters $\mathcal{T}$. The motion ranges of revolute and prismatic joints in the two branches are flipped. To simplify the notation, we just use $\mathcal{R}$ and $\mathcal{T}$ to represent them. More details are provided in Section B.2.

Since the two branches solve dual problems, we can naturally construct **cycle consistency loss** $\mathcal{L}_{cycle}$ between the branches, which includes both the part segmentation loss and the articulation loss:

$$\mathcal{L}_{\text{cycle}} = \text{BCE}(\hat{\mathcal{J}}_f, \hat{\mathcal{J}}_r) + \text{CE}(\hat{\mathcal{A}}_f, \hat{\mathcal{A}}_r) + \ell_1(\hat{\mathcal{R}}_f, \hat{\mathcal{R}}_r) + \ell_1(\hat{\mathcal{T}}_f, \hat{\mathcal{T}}_r) + \sum_{i=\{0,1\}} \text{CE}(\hat{M}_f^{(s_i)}, \hat{M}_r^{(s_i)}) \tag{3}$$

where the lower script f and r denote the forward-order branch and reverse-order branch, respectively. This cycle consistency loss enforces that the two branches produce consistent predictions, leading to more robust and generalizable performance, as shown in our experiments.

4 EXPERIMENTS

We evaluate the performance of MOTIONART in part segmentation and articulation estimation for both rest and articulated states. Our experiments mainly answer three questions: (1) whether two-state reasoning improves the model generalization over single-state reasoning; (2) whether the network achieves consistent predictions between two states and whether the Siamese framework further improves the consistency; and (3) whether the recovered articulation is good enough to be exported as a URDF asset for downstream robot simulation. We first describe the evaluation protocol, then report comparisons with state-of-the-art methods, ablations, and an application in RoboTwin.

Table 1: **Evaluation on rest and fully articulated states.** For each dataset we report rest-pose segmentation (gIoU, part-wise Chamfer distance PC, and mIoU) and articulated geometry (gIoU, PC, and whole-object Chamfer distance OC). All metrics are computed with penalties applied to unmatched parts. –: method is not applicable for the articulated state. 1@10: best sample out of 10 predictions per instance. Colors: **best** and **second best**.

Method	Lightwheel						PartNet-Mobility					
	Rest-Pose Seg.			Articulated States			Rest-Pose Seg.			Articulated States		
	gIoU↑	PC↓	mIoU↑	gIoU↑	PC↓	OC↓	gIoU↑	PC↓	mIoU↑	gIoU↑	PC↓	OC↓
P3SAM Ma et al. (2025)	0.122	0.177	0.411	–	–	–	-0.116	0.261	0.267	–	–	–
SINGAPO Liu et al. (2025a)	-0.097	0.234	0.273	-0.102	0.329	0.018	0.262	0.124	0.468	0.255	0.184	0.046
SINGAPO Liu et al. (2025a) (1@10)	-0.050	0.221	0.297	-0.056	0.261	0.019	0.271	0.117	0.471	0.264	0.168	0.041
Articulate AnyMesh Qiu et al. (2025)	0.172	0.190	0.452	0.158	0.237	0.010	0.383	0.104	0.542	0.378	0.251	0.022
Particulate Li et al. (2026b)	0.332	**0.168**	0.576	0.305	0.208	0.009	0.880	**0.003**	0.884	0.843	0.022	**0.003**
MOTIONART (Single)	**0.489**	**0.156**	**0.578**	**0.474**	**0.194**	**0.003**	**0.924**	**0.002**	**0.936**	**0.888**	**0.007**	**0.001**
MOTIONART (Siamese)	**0.544**	**0.156**	**0.647**	**0.512**	**0.188**	**0.005**	**0.931**	**0.003**	**0.939**	**0.890**	**0.005**	**0.001**

4.1 EXPERIMENTAL SETUP

Training Datasets. We train MOTIONART on the PartNet-Mobility Mo et al. (2019) and GRScenes Wang et al. (2024) datasets. For PartNet-Mobility, we follow the train-test split of SINGAPO Liu et al. (2025a); for GRScenes, we use all articulated assets for training. We set the maximum number of parts to $K_{\max} = 16$ and discard objects with more than 16 articulated parts, yielding 3,800 training objects across 50 categories.

Testing Datasets. For evaluation, we use the PartNet-Mobility test split, which contains 7 common articulated-object categories, and the Lightwheel benchmark Lightwheel (2025), which contains 243 high-quality articulated objects spanning 14 categories. For every object, we sample two distinct physical states and convert them into a pair of unorganized point clouds. Since the two point clouds are sampled independently, the model does not rely on explicit point correspondence between the states.

Implementation details. Each input state is represented by a point cloud sampled from the corresponding mesh. Following Particulate Li et al. (2026b), we augment point coordinates with surface normals and PartField features. At test time, MOTIONART uses a single forward branch to predict the part masks, kinematic tree, motion type, joint axes, and motion ranges, *i.e.*, the reverse-order branch and cycle-consistency objective are used only during training. Training the main model takes approximately three days on 4 NVIDIA H100 GPUs. More details can be found in Section B.1.

Metrics. We evaluate both part recovery and articulated reconstruction quality. For evaluation on the rest state, we report generalized IoU (gIoU), part-wise Chamfer distance (PC), and mean IoU (mIoU) between predicted and ground-truth parts. For evaluation on the articulated state, we actuate the predicted parts using the predicted joint parameters and report part-wise gIoU, PC, and the whole-object Chamfer distance (OC). The metrics in the articulated state are usually lower than in the rest state because errors in segmentation, kinematic hierarchy, joint axes, and motion range accumulate after actuation. Unless otherwise stated, we use the penalty protocol for unmatched/missing parts as in Li et al. (2026b), which discourages methods from producing visually plausible but incomplete decompositions.

4.2 MAIN RESULTS

Part segmentation at the rest state. Table 1 (**Res-Pose Seg.**) shows the quality of part decomposition in the rest state. On both PartNet-Mobility and Lightwheel, MOTIONART achieves the best gIoU (0.544 *vs.* 0.322), PC (0.156 *vs.* 0.168), and mIoU (0.647 *vs.* 0.576) among the compared methods, indicating that the second-observed state helps the network identify which geometry should move as a rigid part. This improvement is especially important for articulated objects whose static geometry is ambiguous, such as cabinets with visually similar panels or appliances whose movable components are flush with the body. As shown in Figure 5, compared with single-state feed-forward baselines, the

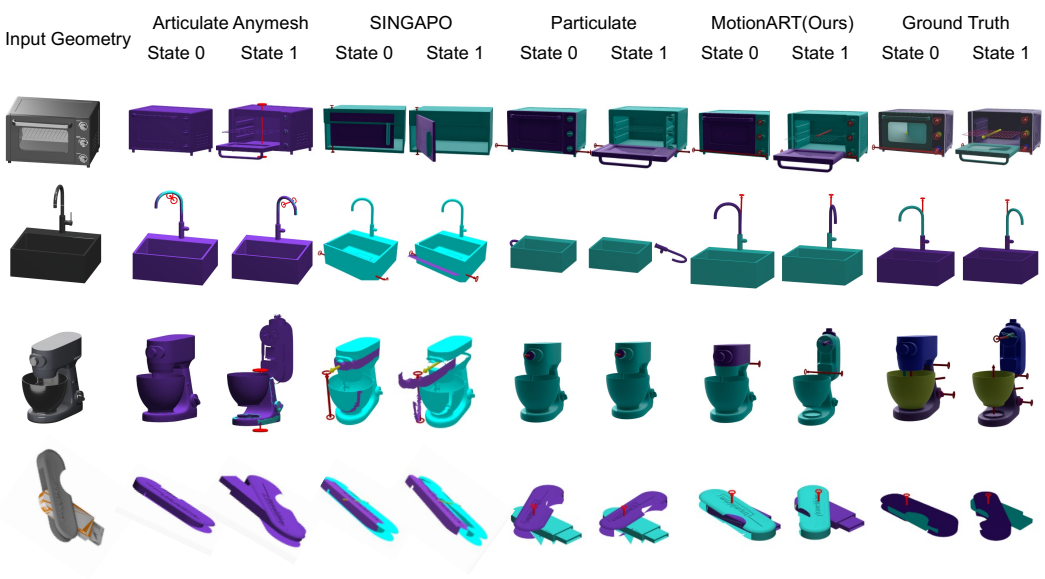

Figure 5: Qualitative comparison of articulation prediction. We visualize the articulated object in both the rest and fully articulated states. Given the same object, MOTIONART predicts more precise part segmentation and better articulation motion on both states than the state-of-the-art methods.

two-state formulation provides direct evidence of rigid part segmentation from relative motion rather than forcing the model to infer articulation only from category-level priors as in Li et al. (2026b).

Articulated-state reconstruction. Table 1 (**Articulated State**) evaluates whether the recovered articulation remains correct after applying the predicted motion. MOTIONART achieves the best performance across all articulated-state metrics, including whole-object Chamfer distance (reducing it from 0.09 to 0.03, a 66% improvement). Because this evaluation actuates each predicted joint to its maximum extent, the result validates more than static segmentation: the predicted kinematic structure, motion type, axis, and range must be mutually consistent. The gap between rest-state and articulated-state performance also reveals why the evaluation should include motion-based metrics; a method can segment parts reasonably while still producing an incorrect joint that fails after actuation. Figure 5 visualizes representative predictions across object categories. All state-of-the-art methods struggle with the "switch" on "microwave", the "arm" of the "Mixer". MOTIONART better captures the motion of these small, protruding parts, which are often visually ambiguous in the rest state but become more distinguishable when observing how they move. This supports the quantitative trend: observing how geometry changes gives the model a more reliable cue for separating functional parts and predicting their motion.

4.3 ABLATION STUDY

We conduct thorough ablations to understand the contribution of different design choices in MOTIONART. Results are summarized in Table 2 and discussed in detail below.

Importance of Motion States. Learning from different states serves as a cornerstone of our method, which formulates articulation recovery as a corresponding motion matching problem. To measure out motion evidence, we compare the default two-state model with a single-state variant. When only one state is observed, the quantitative results drop significantly: gIoU drops from 0.396 to 0.305 on the rest state and from 0.382 to 0.293 on the articulated state (nearly 25% drop), confirming that the second state provides critical evidence for part segmentation and articulation inference.

Ablations on Number of States. Our default model uses two states, but the architecture can be flexibly extended to more states. To verify whether more states can further contribute to performance, we train a multi-state variant. The results show that more states do improve performance, with the

Table 2: **Ablation study on model design choices.** The ablation is done using a subset of training and evaluation data. We report reconstruction quality on both the rest and articulated states, together with inference time. Best results are highlighted in **bold**.

Method	Rest state			Articulated state			Time
	gIoU↑	PC↓	mIoU↑	gIoU↑	PC↓	OC↓	
Single branch							
1-state	0.305	0.096	0.480	0.293	0.144	0.008	7s
2-state (default)	0.396	0.083	0.531	0.382	0.127	**0.004**	10s
4-state	0.440	0.074	0.565	0.424	0.092	**0.004**	15s
8-state	0.456	**0.073**	0.574	**0.447**	**0.088**	**0.004**	18s
w/o attn. mask	0.386	0.093	0.540	0.371	0.134	0.007	10s
Siamese							
w/o cycle loss	0.419	0.085	0.560	0.403	0.127	0.005	10s
final model	**0.462**	0.074	**0.578**	0.445	0.112	**0.004**	10s

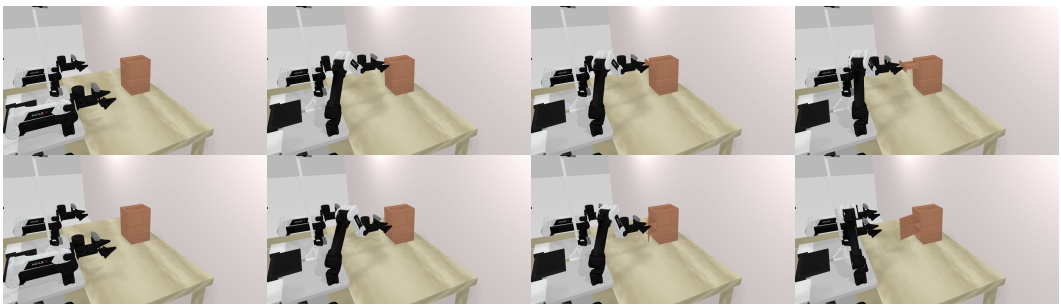

Figure 6: **RoboTwin application with generated URDF assets.** We export MOTIONART's predicted articulated structures as URDF assets and import them into RoboTwin. Each row shows four snapshots of contact-based manipulation: a drawer asset (top) and a door asset (bottom) are opened by a robot arm using the predicted movable link, joint axis, and motion range.

4-state model achieving 0.424 gIoU on the articulated state, and the 8-state model further improved to 0.447, but with diminishing returns and increased inference time.

Ablations on Masked attention. The attention mask further improves segmentation quality by encouraging part queries to focus on their activated regions across transformer layers.

Ablations on Siamese Network. We further demonstrate that the Siamese architecture, along with the cycle-consistency loss, improves the consistency of predictions between the two states. The Siamese model achieves the best or second-best performance compared with the single-branch models, even though the single-branch model with 8 states has more input information. However, removing the cycle-consistency loss from the Siamese model leads to a significant drop in performance, especially for articulated states, indicating that the cycle loss is crucial for stabilizing part identity and motion direction when the state order is reversed.

4.4 APPLICATION

We further test whether the predicted articulation can be exported as a usable robot-simulation asset. To this end, we convert the outputs to a URDF. As shown in Figure 6, the generated drawer and door assets can be loaded into RoboTwin and opened by a robot arm through contact. For the robot to open the generated asset, the predicted movable part, joint axis, and motion range must remain mutually consistent under physics-based interaction. The RoboTwin result, therefore, provides a compact qualitative validation that two-state geometry can support not only offline articulation recovery but also downstream manipulation of generated articulated assets.

5 CONCLUSION

We presented MOTIONART, a feed-forward framework for inferring articulated 3D structure from *two* observed states of the same object. It departs from the common feed-forward methods, which typically learns articulation from a *single* state and relies heavily on category-level priors to infer part motion. It achieves state-of-the-art performance on both rest-state part segmentation and articulated-state reconstruction, with a single forward pass and no post-processing. The ablations further validate the importance of motion evidence, siamese paired learning, and cycle-consistency training. Beyond metrics, we also demonstrate a direct application in which the predicted articulated object is exported as a URDF asset and interacts with a robot arm in the RoboTwin simulator.

AI USE STATEMENT

TODO before submission: Replace this placeholder with the authors' ICLR 2027 AI use disclosure. Describe every required or recommended use of generative AI, how the outputs were checked, and the authors' responsibility for the final paper, code, claims, and artifacts.

REFERENCES

Hao Ai, Wenjie Chang, Jianbo Jiao, Ales Leonardis, and Ofek Eyal. Articulation in motion: Prior-free part mobility analysis for articulated objects by dynamic-static disentanglement, 2026. URL https://arxiv.org/abs/2603.02910.

Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *ECCV*, pp. 213–229. Springer, 2020.

Yuchen Che, Ryo Furukawa, and Asako Kanezaki. Op-align: Object-level and part-level alignment for self-supervised category-level articulated object pose estimation. In *ECCV*, pp. 72–88. Springer, 2024.

Chuhao Chen, Isabella Liu, Xinyue Wei, Hao Su, and Minghua Liu. Freeart3d: Training-free articulated object generation using 3d diffusion. *arXiv preprint arXiv:2510.25765*, 2025a.

Honghua Chen, Yushi Lan, Yongwei Chen, and Xingang Pan. Artilatent: Realistic articulated 3d object generation via structured latents. In *SIGGRAPH Asia*, 2025b.

Minghao Chen, Jianyuan Wang, Roman Shapovalov, Tom Monnier, Hyunyoung Jung, Dilin Wang, Rakesh Ranjan, Iro Laina, and Andrea Vedaldi. Autopartgen: Autogressive 3d part generation and discovery. In *NeurIPS*, 2025c.

Zoey Chen, Aaron Walsman, Marius Memmel, Kaichun Mo, Alex Fang, Karthikeya Vemuri, Alan Wu, Dieter Fox, and Abhishek Gupta. Urdformer: A pipeline for constructing articulated simulation environments from real-world images. *RSS*, 2024.

Bowen Cheng, Ishan Misra, Alexander G. Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention mask transformer for universal image segmentation. In *CVPR*, pp. 1290–1299, June 2022.

Daoyi Gao, Yawar Siddiqui, Lei Li, and Angela Dai. Meshart: Generating articulated meshes with structure-guided transformers. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 618–627, 2025.

Google DeepMind. Gemini image generation (pro), 2025. URL https://deepmind.google/models/gemini-image/pro/.

Pradyumn Goyal, Dmitry Petrov, Sheldon Andrews, Yizhak Ben-Shabat, Hsueh-Ti Derek Liu, and Evangelos Kalogerakis. Geopard: Geometric pretraining for articulation prediction in 3d shapes. In *ICCV*, pp. 9332–9341, October 2025.

Lijun Guo, Haoyu Zhao, Xingyue Zhao, Rong Fu, Linghao Zhuang, Siteng Huang, Zhongyu Li, and Hua Zou. Articulat3d: Reconstructing articulated digital twins from monocular videos with geometric and motion constraints. *arXiv preprint arXiv:2603.11606*, 2026.

Zhao Huang, Boyang Sun, Alexandros Delitzas, Jiaqi Chen, and Marc Pollefeys. React3d: Recovering articulations for interactive physical 3d scenes. *IEEE Robotics and Automation Letters*, 11(5): 5954–5961, 2026. doi: 10.1109/LRA.2026.3674028.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM TOG*, 42(4):1–14, 2023.

Long Le, Jason Xie, William Liang, Hung-Ju Wang, Yue Yang, Yecheng Jason Ma, Kyle Vedder, Arjun Krishna, Dinesh Jayaraman, and Eric Eaton. Articulate-anything: Automatic modeling of articulated objects via a vision-language foundation model. In *ICLR*, 2025.

Jiahui Lei, Congyue Deng, William B Shen, Leonidas J Guibas, and Kostas Daniilidis. Nap: Neural 3d articulated object prior. *NeurIPS*, 36:31878–31894, 2023.

Chengshu Li, Fei Xia, Roberto Martín-Martín, Michael Lingelbach, Sanjana Srivastava, Bokui Shen, Kent Elliott Vainio, Cem Gokmen, Gokul Dharan, Tanish Jain, et al. igibson 2.0: Object-centric simulation for robot learning of everyday household tasks. In *Conf. Robot. Learn.*, 2021.

Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Wensi Ai, Benjamin Jose Martinez, et al. Behavior-1k: A human-centered, embodied ai benchmark with 1, 000 everyday activities and realistic simulation. *CoRR*, 2024.

Haitian Li, Haozhe Xie, Junxiang Xu, Beichen Wen, Fangzhou Hong, and Ziwei Liu. Monoart: Progressive structural reasoning for monocular articulated 3d reconstruction. *arXiv preprint arXiv:2603.19231*, 2026a.

Ruining Li, Yuxin Yao, Chuanxia Zheng, Christian Rupprecht, Joan Lasenby, Shangzhe Wu, and Andrea Vedaldi. Particulate: Feed-forward 3d object articulation. In *CVPR*, 2026b.

Zhe Li, Xiang Bai, Jieyu Zhang, Zhuangzhe Wu, Che Xu, Ying Li, Chengkai Hou, and Shanghang Zhang. Urdf-anything: Constructing articulated objects with 3d multimodal language model, 2025. URL https://arxiv.org/abs/2511.00940.

Lightwheel. Lightwheel sim-ready assets: High-quality usd assets for nvidia isaac sim. GitHub repository, 2025. URL https://github.com/LightwheelAI/Lightwheel-simready-asset. 259 robotics simulation assets under CC BY-NC 4.0.

Jiayi Liu, Ali Mahdavi-Amiri, and Manolis Savva. Paris: Part-level reconstruction and motion analysis for articulated objects. In *ICCV*, pp. 352–363, 2023.

Jiayi Liu, Hou In Ivan Tam, Ali Mahdavi-Amiri, and Manolis Savva. Cage: Controllable articulation generation. In *CVPR*, pp. 17880–17889, 2024.

Jiayi Liu, Denys Iliash, Angel X Chang, Manolis Savva, and Ali Mahdavi Amiri. Singapo: Single image controlled generation of articulated parts in objects. In *ICLR*, 2025a.

Yu Liu, Baoxiong Jia, Ruijie Lu, Chuyue Gan, Huayu Chen, Junfeng Ni, Song-Chun Zhu, and Siyuan Huang. Videoartgs: Building digital twins of articulated objects from monocular video. *arXiv preprint arXiv:2509.17647*, 2025b.

Yu Liu, Baoxiong Jia, Ruijie Lu, Junfeng Ni, Song-Chun Zhu, and Siyuan Huang. Artgs: Building interactable replicas of complex articulated objects via gaussian splatting. *arXiv preprint arXiv:2502.19459*, 2025c.

Ruijie Lu, Yu Liu, Jiaxiang Tang, Junfeng Ni, Yuxiang Wang, Diwen Wan, Gang Zeng, Yixin Chen, and Siyuan Huang. Dreamart: Generating interactable articulated objects from a single image, 2025. URL https://arxiv.org/abs/2507.05763.

Changfeng Ma, Yang Li, Xinhao Yan, Jiachen Xu, Yunhan Yang, Chunshi Wang, Zibo Zhao, Yanwen Guo, Zhuo Chen, and Chunchao Guo. P3-sam: Native 3d part segmentation, 2025. URL https://arxiv.org/abs/2509.06784.

B Mildenhall, PP Srinivasan, M Tancik, JT Barron, R Ramamoorthi, and R Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020.

Kaichun Mo, Shilin Zhu, Angel X. Chang, Li Yi, Subarna Tripathi, Leonidas J. Guibas, and Hao Su. PartNet: A large-scale benchmark for fine-grained and hierarchical part-level 3D object understanding. In *CVPR*, June 2019.

OpenAI. Chatgpt images 2.0, 2026. URL https://openai.com/index/introducing-chatgpt-images-2-0/.

William Peebles and Saining Xie. Scalable diffusion models with transformers. In *ICCV*, pp. 4195–4205, 2023.

Xavier Puig, Eric Undersander, Andrew Szot, Mikael Dallaire Cote, Tsung-Yen Yang, Ruslan Partsey, Ruta Desai, Alexander William Clegg, Michal Hlavac, So Yeon Min, Vladimir Vondrus, Théophile Gervet, Vincent-Pierre Berges, John M. Turner, Oleksandr Maksymets, Zsolt Kira, Mrinal Kalakrishnan, Jitendra Malik, Devendra Singh Chaplot, Unnat Jain, Dhruv Batra, Akshara Rai, and Roozbeh Mottaghi. Habitat 3.0: A co-habitat for humans, avatars and robots. *CoRR*, abs/2310.13724, 2023. URL https://doi.org/10.48550/arXiv.2310.13724.

Xiaowen Qiu, Jincheng Yang, Yian Wang, Zhehuan Chen, Yufei Wang, Tsun-Hsuan Wang, Zhou Xian, and Chuang Gan. Articulate anymesh: Open-vocabulary 3d articulated objects modeling. *arXiv preprint arXiv:2502.02590*, 2025.

Team Seedance, De Chen, Liyang Chen, Xin Chen, Ying Chen, Zhuo Chen, Zhuowei Chen, Feng Cheng, Tianheng Cheng, Yufeng Cheng, et al. Seedance 2.0: Advancing video generation for world complexity. *arXiv preprint arXiv:2604.14148*, 2026.

Licheng Shen, Saining Zhang, Honghan Li, Peilin Yang, Zihao Huang, Zongzheng Zhang, and Hao Zhao. Gaussianart: Unified modeling of geometry and motion for articulated objects. *arXiv preprint arXiv:2508.14891*, 2025.

Chaoyue Song, Jianfeng Zhang, Xiu Li, Fan Yang, Yiwen Chen, Zhongcong Xu, Jun Hao Liew, Xiaoyang Guo, Fayao Liu, Jiashi Feng, et al. Magicarticulate: Make your 3d models articulation-ready. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 15998–16007, 2025.

Jiayi Su, Youhe Feng, Zheng Li, Jinhua Song, Yangfan He, Botao Ren, and Botian Xu. Artformer: Controllable generation of diverse 3d articulated objects. In *CVPR*, pp. 1894–1904, June 2025.

Wei-Cheng Tseng, Hung-Ju Liao, Lin Yen-Chen, and Min Sun. Cla-nerf: Category-level articulated neural radiance field. In *IEEE Int. Conf. Robot. Autom.*, pp. 8454–8460. IEEE, 2022.

Hanqing Wang, Jiahe Chen, Wensi Huang, Qingwei Ben, Tai Wang, Boyu Mi, Tao Huang, Siheng Zhao, Yilun Chen, Sizhe Yang, et al. Grutopia: Dream general robots in a city at scale. *arXiv preprint arXiv:2407.10943*, 2024.

Jiawei Wang, Dingyou Wang, Jiaming Hu, Qixuan Zhang, Jingyi Yu, and Lan Xu. Kinematify: Open-vocabulary synthesis of high-dof articulated objects. *arXiv preprint arXiv:2511.01294*, 2025.

Fangyin Wei, Rohan Chabra, Lingni Ma, Christoph Lassner, Michael Zollhöfer, Szymon Rusinkiewicz, Chris Sweeney, Richard Newcombe, and Mira Slavcheva. Self-supervised neural articulated shape and appearance models. In *CVPR*, pp. 15816–15826, 2022.

Di Wu, Liu Liu, Zhou Linli, Anran Huang, Liangtu Song, Qiaojun Yu, Qi Wu, and Cewu Lu. Reartgs: Reconstructing and generating articulated objects via 3d gaussian splatting with geometric and motion constraints. In *NeurIPS*, 2025a.

Ruiqi Wu, Xinjie Wang, Liu Liu, Chunle Guo, Jiaxiong Qiu, Chongyi Li, Lichao Huang, Zhizhong Su, and Ming-Ming Cheng. Dipo: Dual-state images controlled articulated object generation powered by diverse data. *arXiv preprint arXiv:2505.20460*, 2025b.

Zhuangzhe Wu, Yue Xin, Chengkai Hou, Minghao Chen, Yaoxu Lyu, Jieyu Zhang, and Shanghang Zhang. Urdf-anything+: Autoregressive articulated 3d models generation for physical simulation, 2026. URL https://arxiv.org/abs/2603.14010.

Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, et al. Sapien: A simulated part-based interactive environment. In *CVPR*, pp. 11097–11107, 2020.

What Reveals 3D Articulation? Learning from Motion

Supplementary Material

A Related Work

We provide a comparison table of related works as shown in Table 3.

	Input Modality					Output Properties							Inf.
Method	Text	Vision	Kinematic Chain	**3D**	Number	BBox	**3D**	Segmentation	Structure	Axis	Range	1^+	**Time**
Optimization-based Methods													
PARIS Liu et al. (2023)	–	✓	–	–	2	–	✓	✓	✓	✓	✓	–	N/A
AIM Ai et al. (2026)	–	✓	–	✓	2	–	✓	✓	✓	✓	✓	✓	N/A
Learning-based Methods													
GEOPARD Goyal et al. (2025)	–	–	–	✓†	1	–	✓	–	✓	✓	–	✓	N/A
MeshArt Gao et al. (2025)	–	$(✓)^o$	–	$(✓)^o$	0	✓	✓	✓	–	✓	✓	✓	N/A
PARTICULATE Li et al. (2026b)	–	–	–	✓	1	–	✓	✓	✓	✓	✓	✓	~10s
URDF-Anything Li et al. (2025)	✓	✓	–	✓	1	–	✓	✓	✓	✓	✓	✓	N/A
Generation-based Methods													
SINGAPO Liu et al. (2025a)	–	✓	–	–	1	✓	$(✓)^*$	✓	✓	✓	✓	✓	~10s
CAGE Liu et al. (2024)	✓	–	✓	–	1	✓	$(✓)^*$	✓	✓	✓	–	✓	N/A
Kinematify Wang et al. (2025)	$(✓)^o$	$(✓)^o$	–	–	$(1)^o$	–	✓	✓	✓	✓	–	✓	~20m
Articulate-Anything Le et al. (2025)	✓	$(✓)^o$	–	–	$(1^+)^o$	–	$(✓)^*$	✓	✓	✓	✓	–	~10m
FreeArt3D Chen et al. (2025a)	–	✓	✓	–	6	–	✓	✓	✓	✓	–	–	~10m
Articulate AnyMesh Qiu et al. (2025)	–	–	–	✓	1	–	✓	✓	✓	✓	✓	✓	N/A
DreamArt Lu et al. (2025)	–	✓	–	–	1	–	✓	✓	–	✓	✓	–	N/A
URDF-Anything+ Wu et al. (2026)	–	✓	–	✓	1	–	✓	✓	✓	✓	✓	✓	N/A
ArtiLatent Chen et al. (2025b)	–	$(✓)^o$	–	–	$(1)^o$	–	✓	✓	✓	✓	✓	✓	~30s
ArtFormer Su et al. (2025)	✓	$(✓)^o$	–	–	$(1)^o$	✓	✓	✓	✓	✓	✓	✓	N/A
Ours	–	–	–	✓	2	–	✓	✓	✓	✓	✓	✓	**~10s**

Table 3: **Related work overview on articulated object modeling.** We summarize input modalities, the number of observed articulation configurations, and output properties for various methods. Icons represent: Text, Vision(e.g., image or video), Kinematic Chain, **3D** Geometry(e.g., mesh or point cloud), Number of observed articulation configurations. State counts are modality-agnostic: image, video, mesh, and point-cloud observations are counted by articulation configuration. 0 denotes generation without an observed object state; $(\cdot)^o$ denotes optional conditioning, e.g., $(1)^o$ for one optional state and $(1^+)^o$ for one or more optional states. Output columns show: BBox, **3D** Geometry(e.g., mesh or point cloud), Segmentation, Structure, → Axis, ↕ Range, 1^+ Multi-joint support. $(^*)$ denotes retrieval-based geometry. $(^o)$ indicates optional conditional inputs or outputs. $(^\dagger)$ requires pre-segmented geometry. N/A indicates the inference time is not reported in the original paper.

B Implementation Details

B.1 Data Processing

We preprocess articulated assets from URDF and USD sources into a unified representation for two-state articulation learning. The pipeline first converts articulated objects into normalized meshes

with part-level motion annotations, then samples point clouds from the exported meshes to cache training supervision.

URDF processing. For URDF assets, we parse each object's kinematic tree and mesh references, then export two canonical articulation states. Each processed object is written to an object-level directory, where `pos_000` corresponds to the all-zero joint state and `pos_001` corresponds to the all-one joint state. For each state, the links are transformed by forward kinematics and merged into a single mesh. We filter out invalid assets that contain no valid mesh, only one articulated part, or more than the maximum allowed number of parts. In our preprocessing, we set this maximum to 16. To keep the two states geometrically comparable, we compute a shared normalization transform over all exported states of the same object and apply it consistently to every posed mesh. The same normalization is applied to motion annotations: prismatic ranges are scaled, and revolute or prismatic axes are shifted into the normalized coordinate frame.

USD processing. For USD assets, we follow the same output convention while extracting articulation information from USD physics joints. We traverse the USD stage, identify revolute, prismatic, and fixed joints, recover parent-child link relationships, and compute each joint axis and anchor in world coordinates. We then solve forward kinematics for the two articulation states, export each posed mesh, and save the same metadata files as in the URDF pipeline. The USD pipeline also normalizes all generated states of an object jointly. After all posed meshes are exported, we compute a shared object-level center and scale, rewrite the normalized OBJ files, and transform the saved Plucker axes and motion ranges accordingly.

Point sampling and cached supervision. After mesh export, we cache point-level supervision from each normalized mesh. For each mesh, we sample surface points with their normals and assign them to the articulated parts using the vertex-to-part labels. To better preserve geometric discontinuities between parts, training samples combine uniformly sampled surface points with points sampled near sharp edges. If no sharp edges are detected, the sampler falls back to uniform surface sampling. For training, each cached point cloud stores the sampled point cloud, normals, point-to-part labels, a binary indicator for sharp-edge samples, the part hierarchy, and per-part motion annotations. This cached representation removes the need to repeatedly load articulated meshes during training and provides a unified supervision format across URDF- and USD-derived objects.

B.2 Training Loss Details

For cycle consistency, we enforce that the Siamese branch produces symmetric predictions. Given the reserve-order states as input, we hope the network produces part segmentation results that are consistent with those of the forward-order branch, but with opposite articulation motion. Therefore, we first swap the order of part segmentation results in the reverse branch from $\{\hat{M}_r^{(s_1)}, \hat{M}_r^{(s_0)}\}$ to $\{\hat{M}_r^{(s_0)}, \hat{M}_r^{(s_1)}\}$, matching the corresponding segmentation masks in forward branch. Then we maintain the axis of articulation motion and flip the predicted revolute range and prismatic range from $\alpha_r = \{\alpha_{\min}, \alpha_{\max}\}, l_r = \{l_{\min}, l_{\max}\}$ to $\dot{\alpha}_r = \{\alpha_{\max}, \alpha_{\min}\}, \dot{l}_r = \{l_{\max}, l_{\min}\}$, representing opposite motion. In this way, the model faithfully learns the underlying physical laws hidden in state changes.

C Additional Experimental Results

Additional Ablations. We provide additional ablation results under two evaluation protocols. The no-penalty protocol reports part-wise metrics after matching predicted and ground-truth parts without penalizing unmatched parts, while the penalty protocol assigns penalties to unmatched predictions or missing ground-truth parts. Reporting both protocols helps separate two effects: how well a model aligns the parts it finds and whether it recovers all functional parts required for a complete articulated asset.

Additional Visualizations. We provide additional visualizations in Figure 8 and in the video attached to the supplementary folder.

Table 4: **Additional ablation results without unmatched-part penalty.** We use the same ablation variants and metrics as in the main ablation table, but compute the part-wise metrics without penalizing unmatched predicted or ground-truth parts.

Method	Rest state			Articulated state			Inf. Time
	gIoU↑	PC↓	mIoU↑	gIoU↑	PC↓	OC↓	
Single branch							
1-state	0.470	0.028	0.536	0.457	0.087	0.008	7s
2-state	0.545	0.023	0.580	0.530	0.077	0.004	10s
4-state	0.576	0.020	0.611	0.560	0.041	0.004	15s
8-state	0.577	0.019	0.627	0.598	0.039	0.004	18s
w/o attn. mask	0.567	0.024	0.602	0.561	0.084	0.007	10s
Siamese							
w/o cycle loss	0.583	0.020	0.619	0.567	0.074	0.005	10s
final model	0.578	0.027	0.617	0.560	0.075	0.004	10s

Table 5: **Additional ablation results with unmatched-part penalty.** This table follows the main ablation protocol, where unmatched predicted or ground-truth parts are penalized in the part-wise metrics. The values are copied from the main ablation table for easier side-by-side comparison with Table 4.

Method	Rest state			Articulated state			Inf. Time
	gIoU↑	PC↓	mIoU↑	gIoU↑	PC↓	OC↓	
Single branch							
1-state	0.305	0.096	0.480	0.293	0.144	0.008	7s
2-state	0.396	0.083	0.531	0.382	0.127	0.004	10s
4-state	0.440	0.074	0.565	0.424	0.092	0.004	15s
8-state	0.456	0.073	0.574	0.447	0.088	0.004	18s
w/o attn. mask	0.386	0.093	0.540	0.371	0.134	0.007	10s
Siamese							
w/o cycle loss	0.419	0.085	0.560	0.403	0.127	0.005	10s
final model	0.462	0.074	0.578	0.445	0.112	0.004	10s

Figure 7: We show one failure case on a hierarchical object with multiple coupled rotation axes.

C.1 Failure Case Analysis

A representative failure mode of MotionART occurs on hierarchical objects with multiple coupled rotation axes, as shown in Figure 7. In these cases, the motion of a child part cannot be explained by an independent joint in a fixed global frame: the child joint axis is itself transformed by the motion of its parent link. When the parent joint rotates, the coordinate frame in which the child joint is defined also rotates, so the observed displacement of the child part is a composition of the parent motion and the child's own local articulation. Such configurations are relatively rare in our current training data, making it difficult for a feed-forward model to learn a reliable prior for this form of nested kinematic dependency.

This failure mode is also sensitive to error accumulation along the predicted kinematic tree. If the parent edge is assigned an inaccurate motion type, axis, pivot, or range, the resulting parent transform changes the reference frame used to interpret all descendant joints. The child prediction must then compensate for both its own articulation error and the residual error inherited from the parent, which can produce substantially larger deviations after forward kinematic actuation than the per-joint prediction error would suggest. Therefore, although two-state geometry provides strong evidence for ordinary articulated motion, deeply coupled multi-axis structures remain challenging when the training distribution contains few comparable examples and when small upstream errors propagate to downstream parts.

D Limitations.

Our current formulation assumes that the two observations depict the same object under compatible states and that the target articulation can be represented by a tree-structured URDF. The method may struggle when the two states differ due to severe reconstruction artifacts, non-rigid deformation, missing geometry, or contacts that hide the moving component.

E Statistical Reporting and Impact

Statistical reporting. For all experimental results, we report a single run under the evaluation protocol. All reported metrics are computed on the same evaluation split for a given benchmark, and ablations change only the stated model component or training objective.

Broader impact. MotionART can help convert observed or generated articulated objects into simulation-ready assets, benefiting robotics, embodied AI, and virtual-environment construction. At the same time, incorrect articulation predictions can lead to unreliable simulated interactions, especially if generated assets are used for downstream robot policy training without validation. We therefore view the predicted URDF assets as candidates that should be checked in simulation before

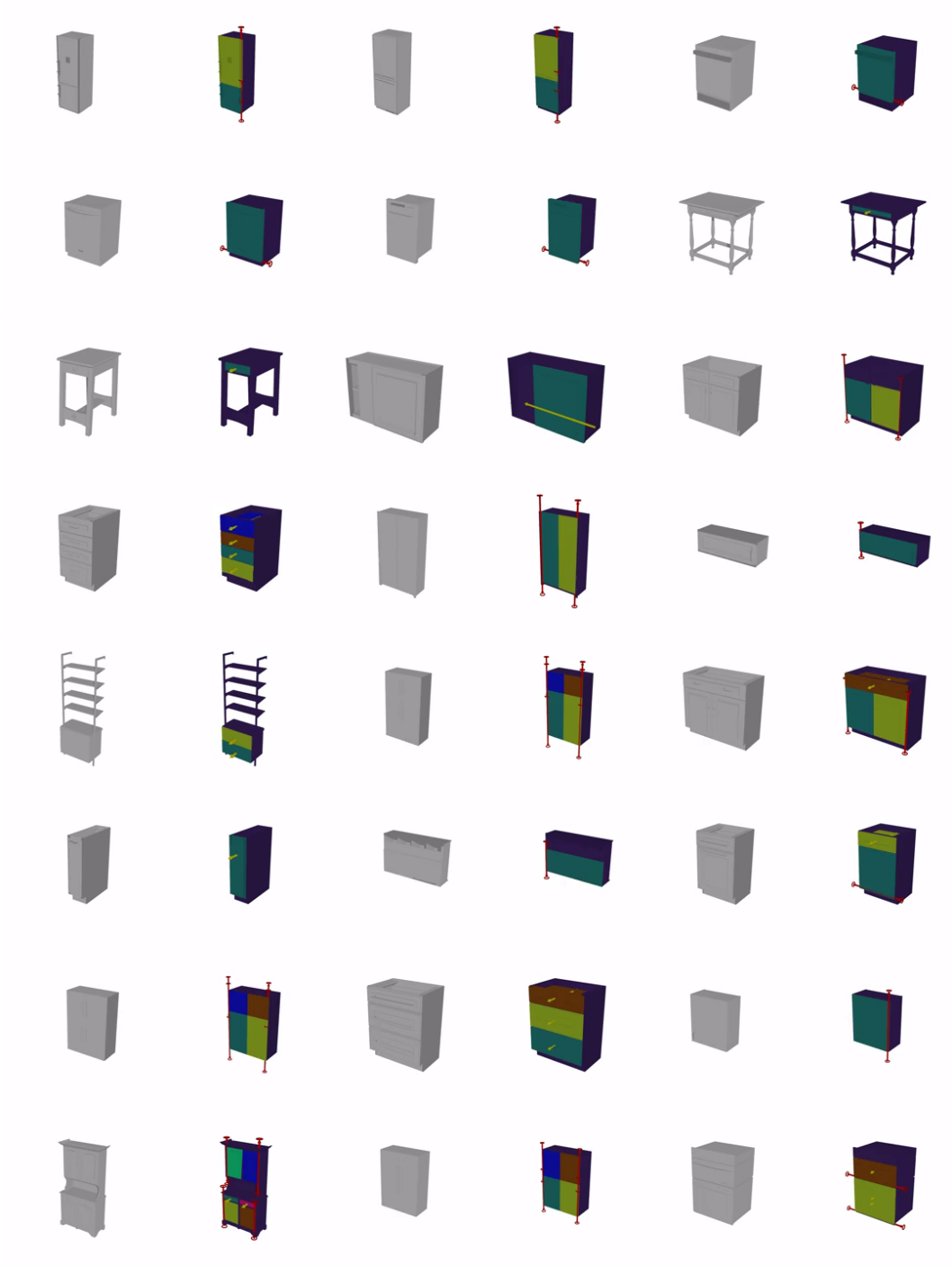

Figure 8: We show more visualization examples on the test set of PartNet-Mobility. Plus, we provide a video version to vividly demonstrate the articulation motion.

deployment on physical robots, and we encourage users to preserve provenance information for generated or converted assets.